

1. Hello World

```
GNU nano 8.7.1 nikhi
#include <stdio.h>

int main() {
    printf("Hello Linux\n");
    return 0;
}
```

```
nikhil@nikhil:~$ nano nikhil.c
nikhil@nikhil:~$ nano nikhil1.c
nikhil@nikhil:~$ gcc nikhil1.c -onikhil1
nikhil@nikhil:~$ ./nikhil1
Hello Linux
nikhil@nikhil:~$ |
```

2. Read User Name

```
GNU nano 8.7.1 nikhil2.c
#include <stdio.h>

int main() {
    char name[30];
    printf("Enter your name: ");
    scanf("%s", name);
    printf("Welcome %s\n", name);
    return 0;
}|
```

```
nikhil@nikhil:~$ nano nikhil2.c
nikhil@nikhil:~$ gcc nikhil2.c -onikhil2
nikhil@nikhil:~$ ./nikhil2
Enter your name: nikhil
Welcome nikhil
nikhil@nikhil:~$ |
```

3. Display Process ID

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Process ID = %d\n", getpid());
    return 0;
}
```

```
nikhil@nikhil:~$ gcc nikhil3.c -onikhil3
nikhil@nikhil:~$ ./nikhil3
Process ID = 802
nikhil@nikhil:~$ nano nikhil3.c
nikhil@nikhil:~$ |
```

4. Basic Fork()

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main() {
    fork();
    printf("Hello\n");
    return 0;
}
```

```
nikhil@nikhil:~$ nano nikhil3.c
nikhil@nikhil:~$ nano nikhil4.c
nikhil@nikhil:~$ gcc nikhil4.c -onikhil4
nikhil@nikhil:~$ ./nikhil4
Hello
Hello
nikhil@nikhil:~$ |
```

5. Parent and Child Branching

```
nikhil@nikhil:~$ nano nikhil5.c
nikhil@nikhil:~$ gcc nikhil5.c -onikhil5
nikhil@nikhil:~$ ./nikhil5
I am Parent Process
I am Child Process
nikhil@nikhil:~$ |
```

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;
    pid = fork();
    if (pid == 0) {
        printf("I am Child Process\n");
    } else {
        printf("I am Parent Process\n");
    }
    return 0;
}
```

6. Display Parent and Child PIDs

```
nikhil@nikhil:~$ nano nikhil5.c
nikhil@nikhil:~$ nano nikhil6.c
nikhil@nikhil:~$ nano nikhil7.c
nikhil@nikhil:~$ gcc nikhil7.c -onikhil7
nikhil@nikhil:~$ ./nikhil7
Parent PID = 943
Child PID = 944
Parent PID = 402
nikhil@nikhil:~$ |
```

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;
    pid = fork();
    if (pid == 0) {
        printf("Child PID = %d\n", getpid());
        printf("Parent PID = %d\n", getppid());
    } else {
        printf("Parent PID = %d\n", getpid());
    }
    return 0;
}
```

7. Wait()

```
nikhil@nikhil:~$ ./nikhil9
Child Running
Parent Running After Child
nikhil@nikhil:~$ |
```

```

GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    pid = fork();
    if (pid == 0) {
        printf("Child Running\n");
    } else {
        wait(NULL);
        printf("Parent Running After Child\n");
    }
    return 0;
}

```

8. Exec()

```

nikhil@nikhil:~$ gcc nikhil44.c -o nikhil44
nikhil@nikhil:~$ ./nikhil44
Before exec()
Documents  exec      fork.c      hello1.c    new.txt     nikhil2.c
Hemasai   exec.c      forkexec    hemasai     nikhil      nikhil3
OSLab      file1.txt   forkexec.c  hemasaifile nikhil.c    nikhil3.c
a.txt      file2.txt   hello       hemasaifile.c nikhil1     nikhil4
b.txt      filename.c  hello.c     hemasaiprograms nikhil1.c   nikhil4.c
data.txt   fork        hello1      input.txt   nikhil2     nikhil44
nikhil@nikhil:~$ |

```

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Before exec()\n");
    execlp("ls", "ls", NULL);
    printf("After exec()\n");
    return 0;
}
```

9. fork() + exec()

```
compilation terminated.
nikhil@nikhil:~$ nano nikhil66.c
nikhil@nikhil:~$ gcc nikhil66.c -onikhil66
nikhil@nikhil:~$ ./nikhil66
Tue Aug 11 13:43:45 UTC 2026
Child Completed
nikhil@nikhil:~$ |
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    pid = fork();
    if (pid == 0) {
        execlp("date", "date", NULL);
    } else {
        wait(NULL);
        printf("Child Completed\n");
    }
    return 0;
}
```